

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Построение и анализ алгоритмов»
Тема: Задача Коммивояжера.

Студент гр. 4345

Наумов М. В.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Задание

В волшебной стране Алгоритмии великий маг, Гамильтон, задумал невероятное путешествие, чтобы связать все города страны закланием процветания. Для этого ему необходимо посетить каждый город ровно один раз, создавая тропу благополучия, и вернуться обратно в столицу, используя минимум своих чародейских сил. Вашей задачей является помощь в прокладывании маршрута с помощью древнего и могущественного алгоритма ветвей и границ.

Карта дорог Алгоритмии перед Гамильтоном представляет собой полный граф, где каждый город соединён магическими порталами с каждым другим. Стоимость использования портала из города в город занимает определённое количество маны, и Гамильтон стремится минимизировать общее потребление магической энергии для закрепления проклятия.

Входные данные:

Первая строка содержит одно целое число N (N — количество городов). Города нумеруются последовательными числами от 0 до $N-1$. Следующие N строк содержат по N чисел каждая, разделённых пробелами, формируя таким образом матрицу стоимостей M . Каждый элемент $M_{i,j}$ этой матрицы представляет собой затраты маны на перемещение из города i в город j .

Выходные данные:

Первая строка: Список из N целых чисел, разделённых пробелами, обозначающих оптимальный порядок городов в магическом маршруте Гамильтона. В начале идёт город 0, с которого начинается маршрут, затем последующие города до тех пор, пока все они не будут посещены. Вторая строка: Число, указывающее на суммарное количество израсходованной маны для завершения пути.

Тестовые данные			^
№ Теста	Входные данные	Выходные данные	
1	3 -1 1 3 3 -1 1 1 2 -1	0 1 2 3.0	
2	4 -1 3 4 1 1 -1 3 4 9 2 -1 4 8 9 2 -1	0 3 2 1 6.0	

Вариант 1. МВиГ: Алгоритм Литтла. Приближённый алгоритм: 2-приближение по МОД (АДО МОД). Замечание к варианту 1. АДО МОД является 2-приближением только для евклидовой матрицы. Начинать обход МОД со стартовой вершины.

Описание алгоритма

1. Точный метод: Метод ветвей и границ с поиском в глубину (DFS)

Данная программа решает задачу коммивояжера с помощью метода ветвей и границ. Основная стратегия заключается в направленном обходе дерева возможных маршрутов с немедленным отсечением тех путей, которые гарантированно хуже уже известного решения. Для принятия решения об отсечении алгоритм постоянно сравнивает верхнюю границу (стоимость текущего лучшего замкнутого маршрута) и нижнюю оценку (оптимистичный прогноз стоимости завершения строящегося пути).

В качестве начального рекорда (верхней границы) изначально принимается бесконечность. Поиск оптимального пути осуществляется с помощью обхода в глубину (DFS), что позволяет минимизировать затраты памяти. Для вычисления нижней оценки (bound) на каждом шаге рекурсии к текущей накопленной стоимости маршрута прибавляется теоретический минимум, необходимый для посещения оставшихся городов.

Эта нижняя оценка формируется на основе вектора минимальных исходящих рёбер графа (min_out): для текущей вершины и для всех ещё не посещённых вершин выбираются их минимальные неотрицательные инцидентные рёбра. Если вычисленная нижняя граница текущей ветви превышает или равна уже найденному лучшему рекорду, ветвь признается бесперспективной и алгоритм выполняет немедленное отсечение (возврат), не заходя вглубь этого поддерева. При достижении базы рекурсии (когда посещены все вершины) проверяется наличие ребра возврата в стартовый город, и в случае успеха фиксируется новый рекорд.

2. Приближённый метод: 2-приближение на основе минимального остовного дерева (АДО МОД)

Для быстрого нахождения субоптимального решения задачи коммивояжера реализован приближённый алгоритм с гарантированной оценкой качества (не более чем в 2 раза хуже оптимального). Работа алгоритма базируется на свойствах метрического пространства графа и строится в три этапа.

На первом этапе из заданной стартовой вершины с помощью жадного алгоритма Прима строится Минимальное Остовное Дерево (МОД). На каждом шаге алгоритм выбирает минимальное по весу допустимое ребро, связывающее множество уже посещённых вершин с ещё не присоединёнными.

На втором этапе для определения порядка обхода вершины полученного дерева сортируются по возрастанию веса исходящих рёбер. Это обеспечивает выполнение условия движения по минимальному доступному ребру из текущей точки.

На третьем этапе выполняется обход дерева в глубину (DFS). Для того чтобы превратить обход дерева в замкнутый маршрут коммивояжера и посетить каждый город ровно один раз, алгоритм использует технику «срезания углов» (shortcuts): если при обходе соседа вершина уже была посещена ранее, алгоритм пропускает её и переходит напрямую к следующему по порядку не посещённому городу. В финале маршрут принудительно замыкается возвратом в стартовую вершину, после чего вычисляется итоговая длина полученного пути.

1. Точный метод: Метод ветвей и границ с поиском в глубину (DFS)

- Начальный этап: Считывание матрицы стоимостей из входного потока и её парсинг требуют $O(N^2)$ операций, где N — количество городов. Расчёт вектора минимальных исходящих рёбер (`min_out`), используемого для дальнейшей оценки границ, выполняется за $O(N^2)$ времени, так как для каждой из N строк матрицы ищется минимальный элемент среди N столбцов.
- Основной цикл: В худшем сценарии (например, при неблагоприятной структуре матрицы весов, когда отсечения не срабатывают) алгоритм совершает полный перебор всех возможных перестановок вершин, что порождает дерево поиска с количеством узлов порядка $O(N!)$.

- Асимптотика узла: В каждом узле дерева на шаге рекурсии выполняется подсчёт нижней границы (проход по не посещённым вершинам за $O(N)$) и перебор её доступных соседей (цикл до $O(N)$). При достижении листа дерева (базы рекурсии) при условии обновления рекорда происходит копирование массива пути, занимающее $O(N)$. Таким образом, обработка одного узла занимает $O(N)$ времени.

- Итог: Общая временная сложность алгоритма в худшем случае составляет $O(N! * N)$. Однако на практике за счёт раннего отсека бесперспективных ветвей с помощью динамической оценки нижней границы (bound) реальное количество исследуемых узлов кардинально меньше.

2. Приближённый метод: 2-приближение на основе минимального остовного дерева (АДО МОД)

В отличие от точного метода, данный алгоритм является полиномиальным и гарантирует быструю работу даже на графах большой размерности.

- Начальный этап: Считывание данных и инициализация матрицы весов занимают $O(N^2)$ операций.

- Построение МОД (Алгоритм Прима): Построение минимального остовного дерева занимает $O(N^2)$ времени. Алгоритм совершает $N-1$ шаг, и на каждом шаге ищет минимальное ребро среди всех связей между уже посещёнными вершинами (их количество растёт до N) и ещё не посещёнными (их количество уменьшается от N). Прimitивный двойной цикл для поиска минимума даёт $O(N^2)$.

- Сортировка списков смежности: Для каждой из N вершин дерева выполняется сортировка списка её соседей. В худшем случае (если дерево выродилось в «звезду», где у одного узла $N-1$ сосед) сортировка займёт $O(N * \log N)$. Для всего графа эта процедура не превышает $O(N^2 * \log N)$ (но это случится, если каждая вершина будет иметь n

соседей, но у нас дерево, поэтому такое невозможно и время сортировки не достигнет таких высот).

- Обход в глубину (DFS): Поиск в глубину по построенному дереву посещает каждую вершину и каждое ребро фиксированное число раз. Поскольку в дереве ровно $N-1$ ребро, сложность DFS составляет $O(N)$. Подсчёт финальной длины пути также требует одного прохода по массиву из $N+1$ элементов, что занимает $O(N)$.

- Итог: Суммарная временная сложность приближённого алгоритма определяется наиболее тяжёлым этапом (построением дерева) и составляет $O(N^2)$ в худшем, среднем и лучшем случаях, что позволяет алгоритму работать практически мгновенно.

Сложность по памяти:

1. Точный метод: Метод ветвей и границ с поиском в глубину (DFS)

- Основной объём памяти: На этапе инициализации основная часть оперативной памяти выделяется под хранение двумерной матрицы смежности графа размерностью $N * N$, что требует $O(N^2)$ памяти.

- Вспомогательные структуры: Массив для отслеживания посещённых городов (visited), массив текущего пути (current_path) и массив для хранения лучшего найденного маршрута (best_path) имеют фиксированную длину N и суммарно занимают $O(N)$ памяти. Вектор минимальных исходящих рёбер (min_out), используемый для расчёта нижних оценок, также требует $O(N)$ пространства.

- Стек рекурсии: В отличие от реализаций, основанных на очередях с приоритетом (где количество хранящихся узлов дерева решений в худшем случае может деградировать до факториала), использование поиска в глубину (DFS) гарантирует строгую локализацию памяти. Максимальная глубина рекурсивного стека жёстко ограничена количеством городов и составляет ровно N уровней. На каждом уровне рекурсии хра-

няться лишь примитивные локальные переменные, что требует $O(1)$ памяти на один вызов. Копирование массива пути при обновлении рекорда выполняется локально и не раздувает стек.

- Итог: Пиковое потребление памяти алгоритмом ветвей и границ полностью определяется размером входных данных и составляет $O(N^2)$, что делает программу крайне нетребовательной к ресурсам даже при глубоком переборе.

2. Приближённый метод: 2-приближение на основе минимального остовного дерева (АДО МОД)

Данный алгоритм также оптимизирован с точки зрения расхода памяти и не требует создания сложных динамических структур.

- Входные данные: Как и в точном методе, хранение матрицы весов графа занимает доминирующий объём памяти, равный $O(N^2)$.

- Структуры для алгоритма Прима и DFS: Вспомогательный массив для отслеживания посещённых вершин при построении остова (`visited_mst`) и аналогичный массив для поиска в глубину (`visited_dfs`) требуют по $O(N)$ памяти.

- Представление дерева: Минимальное остовное дерево (MST) хранится в памяти в виде списка смежности (`mst_adj`). Поскольку в любом дереве с N вершинами количество рёбер всегда строго равно $N-1$, суммарное число элементов во всех списках смежности равно $2(N-1)$. Таким образом, структура графа дерева занимает всего $O(N)$ памяти.

- Выходной маршрут: Хранение результирующего массива вершин (`path`), формируемого в процессе обхода дерева и выполнения `shortcuts`, требует $O(N)$ памяти (максимум $N+1$ элемент с учётом возврата в стартовую точку). Стек вызовов функции `dfs` при обходе дерева ограничен его высотой, которая в худшем случае (вырождение дерева в цепочку) составляет $O(N)$.

- Итог: Общая пространственная сложность приближённого алгоритма составляет $O(N^2)$, где основным и самым ресурсоёмким элементом является исключительно исходная матрица весов.

Структуры данных и ключевые переменные:

1. Точный метод: Метод ветвей и границ с поиском в глубину (DFS)

- `matrix (list)`: Базовая структура данных, представляющая собой двумерный список (матрицу смежности) размерностью $N * N$. Хранит стоимости (веса) переходов между всеми парами вершин (городов). Значение -1 на главной диагонали или в местах отсутствия рёбер означает невозможность прямого перехода.
- `n (int)`: Переменная, хранящая общее количество вершин (городов) в исследуемом графе.
- `min_out (list)`: Одномерный массив длины N , в котором для каждого города заранее вычисляется и хранится минимальный вес исходящего из него допустимого ребра. Используется на каждом шаге рекурсии для быстрого расчёта нижней границы.
- `best_cost (float)`: Переменная верхней границы (рекорда). Хранит стоимость лучшего полного замкнутого маршрута, найденного на данный момент. Инициализируется бесконечностью (`float('inf')`) и служит главным критерием для отсека неперспективных ветвей.
- `best_path (list)`: Одномерный массив, в который копируется самый короткий полный маршрут (последовательность индексов городов) при обновлении значения `best_cost`.
- `current_path (list)`: Динамический массив (длиной N), хранящий последовательность городов текущего строящегося маршрута на данном уровне рекурсии.
- `visited (list)`: Массив логических значений (`True/False`) длины N , используемый для отслеживания состояния вершин. Предотвращает повторное посещение городов внутри одной ветви дерева решений.

- `bound (float)`: Локальная переменная внутри функции `dfs`, хранящая нижнюю оценку (оптимистичный прогноз) стоимости завершения текущего пути. Если `bound >= best_cost`, алгоритм прекращает ветвление.

2. Приближённый метод: 2-приближение на основе минимального остовного дерева (АДО МОД)

- `adj_matrix (list)`: Двумерный список (матрица весов графа) размерностью $N \times N$, хранящий расстояния между городами.
- `start_vertex (int)`: Переменная, определяющая индекс начального города, с которого начинается как построение минимального остовного дерева, так и финальный обход коммивояжера.
- `mst_adj (dict)`: Список смежности, представляющий структуру построенного минимального остовного дерева. Ключами являются индексы вершин, а значениями — списки соединённых с ними соседей в дереве, отсортированные по возрастанию веса ребра.
- `visited_mst (list)`: Массив логических значений длины N , используемый в алгоритме Прима для разделения вершин на те, что уже включены в остовное дерево, и те, которые ещё предстоит присоединить.
- `visited_dfs (list)`: Массив логических значений длины N , используемый при поиске в глубину по построенному дереву для исключения повторного захода в уже посещённые вершины (реализация механизма `shortcuts`).
- `path (list)`: Одномерный массив, в который последовательно записывается результирующий маршрут коммивояжера. В конце алгоритма в него принудительно добавляется `start_vertex` для замыкания пути.
- `total_length (float)`: Переменная, в которой аккумулируется суммарный вес всех рёбер, вошедших в финальный замкнутый маршрут.

Функции алгоритма:

1. Точный метод: Метод ветвей и границ с поиском в глубину (DFS)

- `dfs(curr_node, current_cost, level)`: Главная рекурсивная функция, реализующая поиск в глубину с одновременным ветвлением и отсечением по методу ветвей и границ. Функция пошагово строит дерево решений, вычисляет нижнюю границу (`bound`) для текущей ветви и выполняет отсечение, если прогноз стоимости хуже текущего рекорда. При достижении полной длины маршрута проверяет условия замыкания пути и обновляет глобальный минимум. Каждое действие функции подробно протоколируется в файл `log.txt`.

- Аргументы:

- `curr_node (int)` — индекс текущего города, в котором находится коммивояжёр.
- `current_cost (float)` — накопленная стоимость пути, пройденного к данному моменту.
- `level (int)` — текущая глубина рекурсии (количество уже посещённых городов).

- `solve()`: Основная функция и точка входа в программу точного метода. Отвечает за открытие файла `log.txt` и первичную запись метаданных. Производит считывание данных из стандартного потока ввода (`sys.stdin`), формирует размерность `n` и двумерный список `matrix`. Выполняет предварительный расчёт вектора `min_out` для каждого города с фиксацией результатов в лог. Инициализирует стартовые массивы, запускает рекурсивный процесс `dfs(0, 0.0, 1)`, после чего закрывает лог-файл и выводит финальный результат (путь и его точную стоимость) в консоль.

- Аргументы: не принимает.

2. Приближённый метод: 2-приближение на основе минимального остовного дерева (АДО МОД)

- `dfs(node, level=1)`: Вспомогательная рекурсивная функция для обхода построенного минимального остовного дерева (MST) в глубину. Реализует механизм `shortcuts` (срезания углов): последовательно обходит отсортированных по весу соседей текущего узла и добавляет в итоговый массив `path` только те города, в которых алгоритм ещё не был. Процесс входа в узлы, обнаружения уже посещённых вершин и возврата назад логируется с иерархическими отступами в `log.txt`.

- Аргументы:

- `node (int)` — индекс текущей вершины дерева.
- `level (int)` — уровень вложенности обхода, используемый исключительно для форматирования отступов в лог-файле.

Тестирование

Результаты тестирования представлены в табл. 1 и 2.

Таблица 1 – Результаты тестирования (Алгоритм МВиГ)

№ п/п	Входные данные	Выходные данные	Комментарии
1.	4 -1 3 4 1 1 -1 3 4 9 2 -1 4 8 9 2 -1	0 3 2 1 6.0	Простой тест с верным ответом
2.	7 -1 12.5 24.1 18.3 35.2 14.8 22.0 12.5 -1 15.6 28.4 42.1 19.5 31.2 24.1 15.6 -1 10.2 33.7 27.4 20.8 18.3 28.4 10.2 -1 25.9 21.1 13.4 35.2 42.1 33.7 25.9 -1 30.5 16.7 14.8 19.5 27.4 21.1 30.5 -1 11.3 22.0 31.2 20.8 13.4 16.7 11.3 -1	0 1 2 3 4 6 5 106.99999999999999	Тест сложнее верным ответом (но с небольшой погрешностью из-за специфики float)
3.	13 -1 9.22 23.32 8.49 16.76 30.0 26.02 13.89 3.61 13.15 21.4 8.06 4.12 9.22 -1 32.2 17.69 25.5 39.2 33.38 13.93 12.73 21.02 30.48 5.1 11.66 23.32 32.2 -1 15.23 15.52 12.17 25.24 27.29 19.72 12.21 3.16 29.0 20.62 8.49 17.69 15.23 -1 10.05 21.63 21.19 18.03 5.0 8.06 13.04 15.65 7.28 16.76 25.5 15.52 10.05 -1 15.26 11.66 28.07	0 1 11 7 9 2 10 5 6 4 3 8 12 117.99999999999999	Сложный тест с верным ответом (но снова с небольшой погрешностью из-за специфики float)

	14.14 16.12 12.37 24.74 17.03 30.0 39.2 12.17 21.63 15.26 -1 19.1 37.64 26.63 22.56 11.05 37.22 28.65 26.02 33.38 25.24 21.19 11.66 19.1 -1 38.83 24.33 27.78 22.2 34.0 27.46 13.89 13.93 27.29 18.03 28.07 37.64 38.83 -1 14.56 15.23 26.93 8.94 11.4 3.61 12.73 19.72 5.0 14.14 26.63 24.33 14.56 -1 10.0 17.8 10.77 3.16 13.15 21.02 12.21 8.06 16.12 22.56 27.78 15.23 10.0 -1 11.7 17.2 9.49 21.4 30.48 3.16 13.04 12.37 11.05 22.2 26.93 17.8 11.7 -1 27.66 19.1 8.06 5.1 29.0 15.65 24.74 37.22 34.0 8.94 10.77 17.2 27.66 -1 8.6 4.12 11.66 20.62 7.28 17.03 28.65 27.46 11.4 3.16 9.49 19.1 8.6 -1	
--	---	--

Таблица 2 – Результаты тестирования (2-приближение по МОД (АДО МОД))

№ п/п	Входные данные	Выходные данные	Комментарии
1.	0 -1 3 4 1 1 -1 3 4 9 2 -1 4 8 9 2 -1	6.00 0 3 2 1 0	Верный результат, совпадающий с точным
2.	0 -1 12.5 24.1 18.3 35.2 14.8 22.0 12.5 -1 15.6 28.4 42.1 19.5 31.2 24.1 15.6 -1 10.2 33.7	135.80 0 1 5 6 3 2 4 0	Тест сложнее с приблизительно верным ответом (не превышает верный более, чем в 2 раза)

	27.4 20.8 18.3 28.4 10.2 -1 25.9 21.1 13.4 35.2 42.1 33.7 25.9 -1 30.5 16.7 14.8 19.5 27.4 21.1 30.5 -1 11.3 22.0 31.2 20.8 13.4 16.7 11.3 -1		
3.	0 -1 9.22 23.32 8.49 16.76 30.0 26.02 13.89 3.61 13.15 21.4 8.06 4.12 9.22 -1 32.2 17.69 25.5 39.2 33.38 13.93 12.73 21.02 30.48 5.1 11.66 23.32 32.2 -1 15.23 15.52 12.17 25.24 27.29 19.72 12.21 3.16 29.0 20.62 8.49 17.69 15.23 -1 10.05 21.63 21.19 18.03 5.0 8.06 13.04 15.65 7.28 16.76 25.5 15.52 10.05 -1 15.26 11.66 28.07 14.14 16.12 12.37 24.74 17.03 30.0 39.2 12.17 21.63 15.26 -1 19.1 37.64 26.63 22.56 11.05 37.22 28.65 26.02 33.38 25.24 21.19 11.66 19.1 -1 38.83 24.33 27.78 22.2 34.0 27.46 13.89 13.93 27.29 18.03 28.07 37.64 38.83 -1 14.56 15.23 26.93 8.94 11.4 3.61 12.73 19.72 5.0 14.14 26.63 24.33 14.56 -1 10.0 17.8 10.77 3.16	142.98 0 8 12 3 9 10 2 5 4 6 11 1 7 0	Сложный тест снова с приблизительно верным ответом

13.15 21.02 12.21 8.06		
16.12 22.56 27.78 15.23		
10.0 -1 11.7 17.2 9.49		
21.4 30.48 3.16 13.04		
12.37 11.05 22.2 26.93		
17.8 11.7 -1 27.66 19.1		
8.06 5.1 29.0 15.65		
24.74 37.22 34.0 8.94		
10.77 17.2 27.66 -1 8.6		
4.12 11.66 20.62 7.28		
17.03 28.65 27.46 11.4		
3.16 9.49 19.1 8.6 -1		

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: fl6.py

```
import sys

def solve():
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    log = open("log.txt", "w", encoding="utf-8")
    log.write("--- Начало работы алгоритма ветвей и границ для TSP ---\n")

    n = int(input_data[0])
    log.write(f"Размерность матрицы (количество городов): {n}\n")

    matrix = []
    idx = 1

    for i in range(n):
        row = []
        for j in range(n):
            row.append(float(input_data[idx]))
            idx += 1
        matrix.append(row)

    log.write("\nСчитанная матрица смежности:\n")
    for i in range(n):
        log.write(f"    Город {i}: {matrix[i]}\n")

    log.write("\n--- Расчет минимальных исходящих рёбер (min_out) ---\n")
    min_out = [float('inf')] * n
    for i in range(n):
        min_val = float('inf')
        for j in range(n):
            if i != j and matrix[i][j] >= 0 and matrix[i][j] < min_val:
                min_val = matrix[i][j]
        min_out[i] = min_val
        log.write(f"    Для города {i} минимальное ребро: {min_val}\n")

    best_cost = float('inf')
    best_path = [0] * n

    current_path = [0] * n
    current_path[0] = 0

    visited = [False] * n
    visited[0] = True

    log.write("\n--- Запуск поиска в глубину (DFS) с оценкой границ ---\n")

    def dfs(curr_node, current_cost, level):
        nonlocal best_cost, best_path

        indent = "  " * level

        log.write(f"{indent}Текущий город: {curr_node}, Уровень: {level}, Те-  
кущая стоимость: {current_cost}\n")
        log.write(f"{indent}Текущий маршрут: {current_path[:level]}\n")
```

```

        if level == n:
            log.write(
                f"{indent}[База рекурсии] Посещены все {n} городов. Проверка
возврата в город 0 из города {curr_node}...\n")
            if matrix[curr_node][0] >= 0:
                total_cost = current_cost + matrix[curr_node][0]
                log.write(f"{indent} -> Успешный возврат. Полная стоимость
замкнутого пути: {total_cost}\n")
                if total_cost < best_cost:
                    log.write(
                        f"{indent} [!!!] Обновление рекорда! Предыдущая луч-
шая стоимость: {best_cost} -> Новая лучшая стоимость: {total_cost}\n")
                    best_cost = total_cost
                    best_path = current_path[:]
                else:
                    log.write(f"{indent} -> Маршрут не эффективнее лучшего
известного ({best_cost}). Пропускаем.\n")
            else:
                log.write(f"{indent} -> Ошибка: Ребро возврата из
{curr_node} в 0 отсутствует (значение -1).\n")
            return

        log.write(
            f"{indent}[Подсчет нижней границы] Базовая стоимость: {cur-
rent_cost} + min_out текущего узла ({min_out[curr_node]})\n")
        bound = current_cost + min_out[curr_node]
        for i in range(n):
            if not visited[i]:
                bound += min_out[i]
                log.write(f"{indent} Добавлен min_out непосещенного города
{i}: +{min_out[i]}\n")

        log.write(f"{indent} -> Итоговая нижняя граница (bound) для ветви:
{bound} (Текущий рекорд: {best_cost})\n")

        if bound >= best_cost:
            log.write(
                f"{indent}[X] ОТСЕЧЕНИЕ ВЕТВИ: Нижняя граница {bound} >= луч-
шей стоимости {best_cost}. Поиск в этом направлении прекращен.\n")
            return

        log.write(f"{indent}[Цикл ветвления] Перебор возможных переходов из
города {curr_node}:\n")
        for i in range(n):
            if not visited[i]:
                if matrix[curr_node][i] >= 0:
                    log.write(
                        f"{indent} -> Город {i} свободен. Переход по ребру
со стоимостью {matrix[curr_node][i]}\n")
                    visited[i] = True
                    current_path[level] = i

                    dfs(i, current_cost + matrix[curr_node][i], level + 1)

                    visited[i] = False
                    log.write(
                        f"{indent}[Шаг назад / Backtracking] Возврат из ветки
города {i}. Восстановление состояния для города {curr_node}.\n")
                else:
                    log.write(f"{indent} -> Город {i} свободен, но путь в
него невозможен (нет ребра).\n")
            else:
                log.write(f"{indent} -> Город {i} пропущен (уже посе-
щен).\n")

```

```

    dfs(0, 0.0, 1)

    log.write("\n--- Алгоритм завершен ---\n")
    log.write(f"Лучший найденный путь: {best_path}\n")
    log.write(f"Минимальная стоимость: {best_cost}\n")
    log.close()

    print(*best_path)
    print(float(best_cost))

if __name__ == "__main__":
    solve()

```

Название файла: fl7.py

```

import sys

input_data = sys.stdin.read().split()

log = open("log.txt", "w", encoding="utf-8")
log.write("--- Начало работы 2-приближенного алгоритма (АДО МОД) для TSP ---\n")

start_vertex = int(input_data[0])
log.write(f"Стартовая вершина для обхода: {start_vertex}\n")

matrix_elements = input_data[1:]
n = int(len(matrix_elements) ** 0.5)
log.write(f"Количество вершин в графе (n): {n}\n")

adj_matrix = []
idx = 0
for i in range(n):
    row = []
    for j in range(n):
        row.append(float(matrix_elements[idx]))
        idx += 1
    adj_matrix.append(row)

log.write("\nСчитанная матрица весов графа:\n")
for i in range(n):
    log.write(f"    Вершина {i}: {adj_matrix[i]}\n")

log.write("\n--- Построение Минимального Остовного Древа (Алгоритм Прима) --\n")
mst_adj = {i: [] for i in range(n)}
visited_mst = [False] * n

visited_mst[start_vertex] = True
log.write(f"Инициализация MST со стартовой вершины: {start_vertex}\n")

for step in range(n - 1):
    log.write(f"\n[Шаг {step + 1}] Поиск минимального ребра для расширения MST:\n")
    min_weight = float('inf')
    u_min, v_min = -1, -1

    for u in range(n):
        if visited_mst[u]:
            for v in range(n):
                if not visited_mst[v] and adj_matrix[u][v] != -1:

```

```

        if adj_matrix[u][v] < min_weight:
            min_weight = adj_matrix[u][v]
            u_min, v_min = u, v

    if u_min != -1 and v_min != -1:
        mst_adj[u_min].append(v_min)
        mst_adj[v_min].append(u_min)
        visited_mst[v_min] = True
        log.write(f"    -> Добавлено ребро ({u_min} <-> {v_min}) с весом
{min_weight}\n")
        log.write(f"    -> Вершина {v_min} добавлена в множество посещенных в
MST\n")
    else:
        log.write("    [!] Ошибка: Не удалось найти связное ребро. Граф несвяз-
ный.\n")

log.write("\n--- Сортировка списков смежности MST по весам ребер ---\n")
for u in range(n):
    log.write(f"    Вершина {u} до сортировки: {mst_adj[u]}\n")
    mst_adj[u].sort(key=lambda v: adj_matrix[u][v])
    log.write(f"    Вершина {u} после сортировки (приоритет минимальным реб-
рам): {mst_adj[u]}\n")

log.write("\n--- Обход дерева в глубину (DFS) для построения маршрута ---\n")
path = []
visited_dfs = [False] * n

def dfs(node, level=1):
    indent = " " * level
    log.write(f"{indent}Вход в вершину {node}\n")

    visited_dfs[node] = True
    path.append(node)
    log.write(f"{indent}    -> Вершина {node} добавлена в итоговый путь
(Shortcuts в действии)\n")

    for neighbor in mst_adj[node]:
        if not visited_dfs[neighbor]:
            log.write(f"{indent}    Идем в непосещенного соседа {neighbor}\n")
            dfs(neighbor, level + 1)
        else:
            log.write(f"{indent}    Сосед {neighbor} уже посещен, пропускаем
его (срезаем угол)\n")

    log.write(f"{indent}Выход из вершины {node}\n")

dfs(start_vertex)

log.write(f"\nЗамыкание маршрута: возврат в стартовую вершину {start_ver-
tex}\n")
path.append(start_vertex)

log.write("\n--- Подсчет итоговой длины построенного пути ---\n")
total_length = 0.0
for i in range(len(path) - 1):
    u = path[i]
    v = path[i + 1]
    step_cost = adj_matrix[u][v]
    total_length += step_cost
    log.write(f"    Переход [{i}]: {u} -> {v} | Стоимость: {step_cost} | Теку-
щая сумма: {total_length}\n")

log.write("\n--- Алгоритм завершен ---\n")
log.write(f"Итоговый путь: {path}\n")

```

```
log.write(f"Полная длина пути: {total_length:.2f}\n")
log.close()

print(f"{total_length:.2f}")
print(*path)
```